

M - N User - to - Kernel Level Multi Threading Library

1. mn_thread.h

- `mn_thread_t` (**User Thread**):
 - This is a lightweight user thread.
 - It doesn't rely on the OS to schedule it.
 - `ucontext_t context` , which stores the CPU register states, stack pointer, and program counter.
- `mn_kernel_thread_t` (**Kernel Thread**):
 - This represents an actual OS thread (using POSIX `pthread_t`).
 - keeps track of which user threads are assigned to it via the `assigned_threads` array
 - and also keeps a track of what user thread is currently running on that kernel thread(on that cpu core).

2. mn_thread.c

`mn_thread_init`

- Initializes global configuration variables like the number of user/kernel threads, time quantum, and burst times.
- It is the very first function called in `main()` of the test files.

`mn_thread_create`

- Prepares a user thread to run. It doesn't execute the thread immediately; it simply sets up the environment.
- walkthrough :
 1. Sets the state to `THREAD_READY` .
 2. Calls `getcontext()` to initialize the context structure.
`getcontext(ucontext_t *ucp)` : Saves the currently active CPU state into the structure.
 3. Allocates memory for a private stack (`malloc(STACK_SIZE)`).
 4. Calls `makecontext()` . `makecontext(ucontext_t *ucp, void (*func)(), int argc, ...)` :
Modifies a context to point to a specific function call.
It tells the context: *"When you start, execute the `wrapper_function` , and pass it the user's `start_routine` and `arg` ."*

`wrapper_function`

- Acts as a middleman. It takes the user's actual function, runs it, and then explicitly calls `mn_thread_exit()` when the user's function finishes.
- The wrapper ensures the thread gracefully terminates and yields the CPU back to the scheduler after clearing the allocated stack

`mn_thread_wait`

- A blocking function that forces a kernel thread to wait until all its assigned user threads reach the `THREAD_TERMINATED` state.
- Called by the kernel thread function (in the test files) just before it finishes its execution to prevent

3. scheduler.c

`mn_thread_yield`

- Voluntarily gives up the CPU core (the kernel thread) and switches execution to the next available user thread in the same kernel group.
- **When it is called:**
 1. Voluntarily by a thread when its time quantum expires (simulating an interrupt).
 2. By `mn_thread_exit` when a thread finishes its job.
- **Step-by-step Walkthrough:**
 1. **State Management:** If the current thread isn't dead (`TERMINATED`), it changes its state back to `THREAD_READY` .
 2. **Next Thread Calculation:** It uses a mathematical formula (`j_value`) to figure out the next thread. the threads are interleaved. For example, if there are 4 kernel threads, K-Thread 0 gets User Threads 0, 4, 8, 12. This math finds the next thread in that specific sequence (Round Robin).
 3. **Termination Check:** It checks if the next thread is terminated. If so, it loops to find a ready one.
 4. **All Dead Fallback:** If *all* user threads assigned to this kernel thread are dead, it uses `swapcontext` to jump back to the `k_context` (the kernel thread's original state).
 5. **Context Switch:** If a valid next thread is found, it updates the state to `THREAD_RUNNING` and uses `swapcontext(&prev, &next)` to pause the current thread and resume the next one.

`mn_thread_exit`

- Cleans up a thread.
- Automatically by the `wrapper_function` when a user thread's main routine completes.
- **Why it is needed:** It marks the thread as `THREAD_TERMINATED` so the scheduler knows to skip it. Crucially, it `free()`s the allocated stack memory to prevent memory leaks, and then forcefully calls `mn_thread_yield()` to pass control to another thread.

4. Test Files

`test_fact.c` (Factorial Calculation)

- **The Goal:** Calculate 15! (factorial of 15) using parallel processing.
- **The Setup:** 6 User Threads (M) running on 3 Kernel Threads (N).
- **The Execution Flow:**
 1. The mathematical range (1 to 15) is split. Each kernel thread is responsible for a chunk (e.g., K-Thread 0 handles 1-5).
 2. The kernel thread's chunk is further split among its 2 assigned user threads (e.g., U-Thread 0 calculates $1 * 2$, U-Thread 3 calculates $3 * 4 * 5$).
 3. `kernel_thread_function` is the entry point for `pthread_create` . It saves its own context (`getcontext`), kicks off its first assigned user thread using `swapcontext` , and then waits for them to

finish using `mn_thread_wait`.

4. Once all U-Threads finish, the K-Thread multiplies their partial products together.
 5. Finally, `main()` multiplies the results from all 3 K-Threads to get the final answer.
- It demonstrates **Data Parallelism**—splitting a large mathematical task into smaller chunks computed simultaneously.

`test_http_sim.c` (HTTP Request Simulator)

- **The Goal:** Simulate a web server handling multiple incoming HTTP requests, using Round-Robin scheduling to ensure no request gets starved of CPU time.
- **The Setup:** 16 User Threads on 4 Kernel Threads. Time quantum = 5, Burst time = 12.
- **The walkthrough execution:**
 1. Instead of calculating math, these threads run a `while` loop simulating work (`burst_remaining > 0`).
 2. Every time the work counter hits a multiple of the `time_quantum` (5), the thread explicitly calls `mn_thread_yield()`.
 3. This forces a "cooperative multitasking" context switch, allowing the next HTTP request assigned to that kernel thread to get some processing time.
 4. This continues in a circle (Round Robin) until all threads have their burst time reduced to 0.
- It demonstrates **Concurrency and Scheduling**. It proves the `yield` logic works correctly, multiplexing many tasks over fewer actual CPU cores without letting any single heavy task block the system.